

код 4.1 — UART: ESP32 ведучий / STM32 ведений

Ведучий ESP32-S3: консоль USB Serial/JTAG, канал UART2 GPIO17/16. Ведений STM32F411: USART1 PA9 TX / PA10 RX, ШІМ PA6 PA7 PB0 PB1. TX/RX перехрещені, спільний GND. 115200 8N1.

vscode-uart-master / main.c — ESP32-S3, Monitor тут

Target esp32s3. Open Folder на цю теку. Monitor на порту USB JTAG/serial debug unit, не на UART-мості.

```
/*
 * Урок 4.1. Ведучий пристрій стенда: ESP32 DevKit.
 *
 * Призначення: приймати команди оператора з консолі USB, передавати їх
 * веденій платі STM32F411 асинхронним каналом USART і контролювати
 * наявність підтвердження.
 *
 * Схема з'єднань:
 *   ESP32 GPIO17 (передавання) --> STM32 A10 / PA10 (приймання)
 *   ESP32 GPIO16 (приймання)  <-- STM32 A9  / PA9  (передавання)
 *   ESP32 GND                  --- STM32 GND
 *
 * Параметри каналу: 115200 бод, 8 біт даних, без контролю парності,
 * один стоповий біт, апаратне керування потоком вимкнене.
 *
 * Протокол узгоджено з прошивкою STM32UartPr:
 * - команда завершується символом переведення рядка '\n';
 * - ведена плата відповідає рядком, що починається з OK або ERR;
 * - після скидання ведена плата надсилає кілька рядків INFO і рядок OK READY;
 * - розбір команд на веденій платі не залежить від регістру символів.
 */

#include <fcntl.h>
#include <inttypes.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#include "driver/uart.h"
#include "esp_log.h"
#include "esp_timer.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

static const char *TAG = "4.1-master";

/* --- Параметри стенда ----- */

#define LINK_UART          UART_NUM_2
#define LINK_TX_GPIO       17
#define LINK_RX_GPIO       16
#define LINK_BAUD          115200

#define CMD_MAX             160
#define REPLY_TIMEOUT_MS   500 /* очікування підтвердження на одну команду */
#define BANNER_TIMEOUT_MS  1500 /* очікування повідомлення після скидання */
#define LINK_ATTEMPTS       2  /* кількість спроб надіслати одну команду */
#define SYNC_SAMPLES       5   /* кількість вимірювань для оцінки зсуву */

/* --- Стан ведучого пристрою ----- */

typedef enum {
    REPLY_OK,          /* підтвердження виконання */
    REPLY_ERR,         /* команду відхилено */
    REPLY_INFO,        /* повідомлення про стан */
    REPLY_UNKNOWN,     /* рядок поза межами протоколу */
} reply_kind_t;

typedef enum {
    LINK_ACK,          /* отримано рядок OK */
    LINK_NACK,         /* отримано рядок ERR */
    LINK_NO_REPLY,     /* відповіді немає */
} link_result_t;

static struct {
    uint32_t sent;
    uint32_t ack;
```

```

uint32_t nack;
uint32_t no_reply;
uint32_t slave_resets;
} s_stats;

/* Оцінка зсуву між шкалою часу веденої плати і власною шкалою ведучого.
Обчислюється командою SYNC і скидається після перезапуску веденої плати. */
static bool s_offset_valid;
static int64_t s_offset_us;

/* --- Робота з рядками ----- */

static void str_upper(char *s)
{
    for (; *s != '\0'; s++) {
        if (*s >= 'a' && *s <= 'z') {
            *s = (char)(*s - 'a' + 'A');
        }
    }
}

static void str_trim(char *s)
{
    size_t len = strlen(s);
    while (len > 0 && (s[len - 1] == ' ' || s[len - 1] == '\t')) {
        s[--len] = '\0';
    }
    size_t start = 0;
    while (s[start] == ' ' || s[start] == '\t') {
        start++;
    }
    if (start > 0) {
        memmove(s, s + start, len - start + 1);
    }
}

/* Перевіряє, що рядок починається саме цим словом, а не лише його літерами. */
static bool starts_with_word(const char *line, const char *word)
{
    size_t n = strlen(word);
    return strncmp(line, word, n) == 0 && (line[n] == '\0' || line[n] == ' ');
}

static reply_kind_t classify(const char *line)
{
    if (starts_with_word(line, "OK")) {
        return REPLY_OK;
    }
    if (starts_with_word(line, "ERR")) {
        return REPLY_ERR;
    }
    if (starts_with_word(line, "INFO")) {
        return REPLY_INFO;
    }
    return REPLY_UNKNOWN;
}

/* --- Канал до веденої плати ----- */

static void link_init(void)
{
    const uart_config_t cfg = {
        .baud_rate = LINK_BAUD,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
        .source_clk = UART_SCLK_DEFAULT,
    };

```

```

};

ESP_ERROR_CHECK(uart_driver_install(LINK_UART, 1024, 512, 0, NULL, 0));
ESP_ERROR_CHECK(uart_param_config(LINK_UART, &cfg));
ESP_ERROR_CHECK(uart_set_pin(LINK_UART, LINK_TX_GPIO, LINK_RX_GPIO,
                             UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE));
}

/* Читає один рядок до символу '\n'. Символ '\r' та порожні рядки відкидає.
   Повертає довжину рядка або -1, якщо час очікування вичерпано. */
static int link_read_line(char *out, size_t cap, int timeout_ms)
{
    const int64_t deadline = esp_timer_get_time() + (int64_t)timeout_ms * 1000;
    size_t len = 0;

    for (;;) {
        const int64_t left_us = deadline - esp_timer_get_time();
        if (left_us <= 0) {
            return -1;
        }

        uint8_t ch;
        if (uart_read_bytes(LINK_UART, &ch, 1, pdMS_TO_TICKS(left_us / 1000 + 1)) != 1) {
            continue;
        }

        if (ch == '\r') {
            continue;
        }
        if (ch == '\n') {
            out[len] = '\0';
            if (len == 0) {
                continue;
            }
            return (int)len;
        }
        if (len + 1 < cap) {
            out[len++] = (char)ch;
        }
    }
}

static void link_send_line(const char *cmd)
{
    uart_write_bytes(LINK_UART, cmd, strlen(cmd));
    uart_write_bytes(LINK_UART, "\n", 1);
    uart_wait_tx_done(LINK_UART, pdMS_TO_TICKS(100));
}

/* Відображає рядок, надісланий веденою платою поза межами діалогу,
   і виявляє факт її перезапуску. */
static void note_slave_line(const char *line)
{
    ESP_LOGI(TAG, "<-- %s", line);

    if (strstr(line, "slave started") != NULL) {
        s_stats.slave_resets++;
        s_offset_valid = false;
        ESP_LOGW(TAG, "ведена плата перезапустилася: режим ШІМ і лічильник часу скинуто");
    }
}

/* Забирає з буфера рядки, що надійшли до надсилання наступної команди. */
static void link_drain(int timeout_ms)
{
    size_t pending = 0;
    if (uart_get_buffered_data_len(LINK_UART, &pending) != ESP_OK || pending == 0) {
        return;
    }

```

```

    }

    char line[CMD_MAX];
    while (link_read_line(line, sizeof(line), timeout_ms) > 0) {
        note_slave_line(line);
    }
}

static link_result_t link_transact_once(const char *cmd, char *reply, size_t cap)
{
    link_drain(20);
    link_send_line(cmd);
    ESP_LOGI(TAG, "--> %s", cmd);

    const int64_t deadline = esp_timer_get_time() + (int64_t)REPLY_TIMEOUT_MS * 1000;

    for (;;) {
        const int left_ms = (int)((deadline - esp_timer_get_time()) / 1000);
        if (left_ms <= 0 || link_read_line(reply, cap, left_ms) < 0) {
            break;
        }

        switch (classify(reply)) {
            case REPLY_OK:
                ESP_LOGI(TAG, "<-- %s", reply);
                return LINK_ACK;
            case REPLY_ERR:
                ESP_LOGI(TAG, "<-- %s", reply);
                return LINK_NACK;
            case REPLY_INFO:
                /* Повідомлення про стан не є відповіддю: очікування триває далі. */
                note_slave_line(reply);
                break;
            default:
                ESP_LOGW(TAG, "<-- %s   (рядок не є підтвердженням)", reply);
                break;
        }
    }

    reply[0] = '\0';
    return LINK_NO_REPLY;
}

static link_result_t link_transact(const char *cmd, char *reply, size_t cap)
{
    s_stats.sent++;

    for (int attempt = 1; attempt <= LINK_ATTEMPTS; attempt++) {
        const link_result_t result = link_transact_once(cmd, reply, cap);

        if (result == LINK_ACK) {
            s_stats.ack++;
            return result;
        }
        if (result == LINK_NACK) {
            s_stats.nack++;
            return result;
        }

        ESP_LOGW(TAG, "відповіді немає протягом %d мс (спроба %d з %d)",
            REPLY_TIMEOUT_MS, attempt, LINK_ATTEMPTS);
    }

    s_stats.no_reply++;
    ESP_LOGE(TAG, "канал не відповідає. Перевірити в такому порядку: перехресне з'єднання "
        "GPIO17->A10 і GPIO16<-A9, спільну землю, живлення веденої плати, "
        "запис програми та швидкість 115200 бод з обох боків");
    return LINK_NO_REPLY;
}

```

```

}

/* Очікує повідомлення веденої плати після скидання. */
static bool wait_for_ready(int timeout_ms)
{
    const int64_t deadline = esp_timer_get_time() + (int64_t)timeout_ms * 1000;
    char line[CMD_MAX];

    for (;;) {
        const int left_ms = (int)((deadline - esp_timer_get_time()) / 1000);
        if (left_ms <= 0 || link_read_line(line, sizeof(line), left_ms) < 0) {
            return false;
        }

        ESP_LOGI(TAG, "<-- %s", line);
        if (starts_with_word(line, "OK") && strstr(line, "READY") != NULL) {
            return true;
        }
    }
}

/* --- Команди ведучого пристрою ----- */

static void print_help(void)
{
    ESP_LOGI(TAG, "команди веденої плати:");
    ESP_LOGI(TAG, "    PWM MAX | PWM OFF | PWM HALF | PWM FADE – режими яскравості");
    ESP_LOGI(TAG, "    TIME   – час веденої плати від моменту скидання");
    ESP_LOGI(TAG, "    STATUS – поточний режим і коефіцієнт заповнення");
    ESP_LOGI(TAG, "    PING   – перевірка каналу");
    ESP_LOGI(TAG, "команди ведучого пристрою:");
    ESP_LOGI(TAG, "    DEMO   – послідовна перевірка всіх режимів");
    ESP_LOGI(TAG, "    SYNC   – оцінка зсуву шкал часу двох плат");
    ESP_LOGI(TAG, "    STATS  – підсумок обміну");
    ESP_LOGI(TAG, "    HELP   – цей перелік");
}

typedef struct {
    const char *cmd;
    int pause_ms;
    const char *comment;
} demo_step_t;

static const demo_step_t s_demo[] = {
    { "PING",      200, "перевірка каналу" },
    { "STATUS",    200, "стан до змін" },
    { "PWM MAX",   900, "гранична яскравість" },
    { "PWM HALF",  900, "половина яскравості" },
    { "PWM FADE",  3000, "плавна зміна яскравості" },
    { "PWM OFF",   400, "вимкнення" },
    { "PWM BRIGHT", 300, "навмисно неправильний аргумент: очікується ERR" },
    { "TIME",      0, "час веденої плати" },
};

static void run_demo(void)
{
    char reply[CMD_MAX];

    ESP_LOGI(TAG, "--- послідовна перевірка стенда ---");

    for (size_t i = 0; i < sizeof(s_demo) / sizeof(s_demo[0]); i++) {
        ESP_LOGI(TAG, "крок %u з %u: %s",
            (unsigned)(i + 1), (unsigned)(sizeof(s_demo) / sizeof(s_demo[0])),
            s_demo[i].comment);

        if (link_transact(s_demo[i].cmd, reply, sizeof(reply)) == LINK_NO_REPLY) {
            ESP_LOGE(TAG, "перевірку припинено: канал не відповідає");
            return;
        }
    }
}

```

```

    }
    if (s_demo[i].pause_ms > 0) {
        vTaskDelay(pdMS_TO_TICKS(s_demo[i].pause_ms));
    }
}

ESP_LOGI(TAG, "--- перевірку завершено ---");
}

static bool parse_local_ms(const char *reply, uint32_t *out)
{
    const char *p = strstr(reply, "LOCAL_MS=");
    if (p == NULL) {
        return false;
    }
    return sscanf(p + strlen("LOCAL_MS="), "%u SCNu32, out) == 1;
}

/*
 * Оцінка зсуву шкал часу. Ведена плата не має засобів узгодження часу і
 * повідомляє лише власний лічильник, тому зсув обчислює ведучий пристрій.
 * За результат береться вимірювання з найменшим часом обміну: у ньому
 * невизначеність моменту формування відповіді найменша.
 */
static void run_sync(void)
{
    char reply[CMD_MAX];
    int64_t best_rtt_us = INT64_MAX;
    int64_t best_offset_us = 0;
    int good = 0;

    ESP_LOGI(TAG, "--- оцінка зсуву шкал часу, вимірювань: %d ---", SYNC_SAMPLES);

    for (int i = 0; i < SYNC_SAMPLES; i++) {
        const int64_t t_send = esp_timer_get_time();
        const link_result_t result = link_transact("TIME", reply, sizeof(reply));
        const int64_t t_rcv = esp_timer_get_time();

        uint32_t slave_ms;
        if (result != LINK_ACK || !parse_local_ms(reply, &slave_ms)) {
            continue;
        }

        const int64_t rtt_us = t_rcv - t_send;
        if (rtt_us < best_rtt_us) {
            best_rtt_us = rtt_us;
            /* Момент формування відповіді приймається за середину обміну. */
            best_offset_us = (int64_t)slave_ms * 1000 - (t_send + rtt_us / 2);
        }
        good++;
        vTaskDelay(pdMS_TO_TICKS(40));
    }

    if (good == 0) {
        ESP_LOGE(TAG, "оцінку не виконано: жодної придатної відповіді");
        return;
    }

    s_offset_us = best_offset_us;
    s_offset_valid = true;

    ESP_LOGI(TAG, "придатних вимірювань: %d з %d", good, SYNC_SAMPLES);
    ESP_LOGI(TAG, "найменший час обміну: %lld мкс", (long long)best_rtt_us);
    ESP_LOGI(TAG, "зсув шкали веденої плати відносно ведучої: %+lld мс",
               (long long)(best_offset_us / 1000));
    ESP_LOGI(TAG, "надалі час веденої плати можна оцінювати без запиту, "
               "похибка оцінки – близько половини часу обміну");
}

```

```

static void print_stats(void)
{
    ESP_LOGI(TAG, "надіслано команд: %" PRIu32, s_stats.sent);
    ESP_LOGI(TAG, "підтверджено ОК: %" PRIu32 ", відхилено ERR: %" PRIu32
        ", без відповіді: %" PRIu32,
        s_stats.ack, s_stats.nack, s_stats.no_reply);
    ESP_LOGI(TAG, "перезапусків веденої плати: %" PRIu32, s_stats.slave_resets);

    if (s_offset_valid) {
        const int64_t estimate_ms = (esp_timer_get_time() + s_offset_us) / 1000;
        ESP_LOGI(TAG, "оцінка часу веденої плати: LOCAL_MS ≈ %lld", (long long)estimate_ms);
    } else {
        ESP_LOGI(TAG, "зсув шкал часу не визначено, виконати команду SYNC");
    }
}

/* --- Консоль оператора ----- */

static void console_init(void)
{
    /* На S3 кабель сидить на USB Serial/JTAG, не на UART0.
       stdin – це саме той порт, у який пише Monitor. */
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);
    const int flags = fcntl(fileno(stdin), F_GETFL, 0);
    fcntl(fileno(stdin), F_SETFL, flags | O_NONBLOCK);
}

/* Читає рядок з консолі з відображенням введених символів. Поки введення
   немає, забирає з каналу рядки, надіслані веденою платою самостійно. */
static int console_read_line(char *out, size_t cap)
{
    size_t len = 0;

    for (;;) {
        const int ch = getchar();
        if (ch == EOF) {
            clearerr(stdin);
            if (len == 0) {
                link_drain(20);
            }
            vTaskDelay(pdMS_TO_TICKS(20));
            continue;
        }

        if (ch == '\r' || ch == '\n') {
            fputs("\r\n", stdout);
            fflush(stdout);
            out[len] = '\0';
            return (int)len;
        }

        if (ch == '\b' || ch == 0x7F) {
            if (len > 0) {
                len--;
                fputs("\b \b", stdout);
                fflush(stdout);
            }
            continue;
        }

        if (ch < 0x20) {
            continue;
        }

        if (len + 1 < cap) {
            out[len++] = (char)ch;
            putchar(ch);
            fflush(stdout);
        }
    }
}

```

```

    }
}

/* --- Точка входу ----- */

void app_main(void)
{
    console_init();
    link_init();

    ESP_LOGI(TAG, "ведучий пристрій стенда, канал до STM32F411");
    ESP_LOGI(TAG, "з'єднання: GPIO%d --> A10, GPIO%d <-- A9, спільна земля",
        LINK_TX_GPIO, LINK_RX_GPIO);
    ESP_LOGI(TAG, "параметри каналу: %d бод, 8 біт даних, без парності, 1 стоповий біт",
        LINK_BAUD);

    if (wait_for_ready(BANNER_TIMEOUT_MS)) {
        ESP_LOGI(TAG, "ведена плата готова до роботи");
    } else {
        ESP_LOGW(TAG, "повідомлення про готовність не отримано за %d мс", BANNER_TIMEOUT_MS);
        ESP_LOGW(TAG, "ведену плату запущено раніше за ведучу або канал несправний, "
            "виконується перевірка каналу");

        char reply[CMD_MAX];
        if (link_transact("PING", reply, sizeof(reply)) == LINK_ACK) {
            ESP_LOGI(TAG, "канал справний, ведена плата працює з моменту попереднього скидання");
        }
    }

    print_help();
    ESP_LOGI(TAG, "введення команди завершується клавішею Enter");

    char line[CMD_MAX];
    char reply[CMD_MAX];

    for (;;) {
        if (console_read_line(line, sizeof(line)) <= 0) {
            continue;
        }

        str_trim(line);
        str_upper(line);
        if (line[0] == '\0') {
            continue;
        }

        if (strcmp(line, "DEMO") == 0) {
            run_demo();
            continue;
        }
        if (strcmp(line, "SYNC") == 0) {
            run_sync();
            continue;
        }
        if (strcmp(line, "STATS") == 0) {
            print_stats();
            continue;
        }
        if (strcmp(line, "HELP") == 0) {
            print_help();
            /* Перелік запитується також у веденої плати: він показує,
               яка саме версія програми записана в неї. */
        }

        link_transact(line, reply, sizeof(reply));
    }
}

```

STM32UartPr / Src/app.c — STM32, без монітора

CubeIDE, ST-LINK. main.c лише викликає App_*. Відповіді йдуть на ESP32.

```
#include "app.h"
#include "main.h"

#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Довжина рядка команди разом із завершальним нулем. */
#define UART_LINE_MAX 96

/* Період ШИМ: лічильник TIM3 від 0 до 999, тобто 1000 кроків. */
#define PWM_ARR 999U
/* Скважність 100 % і 50 % у тих самих кроках. */
#define PWM_MAX_DUTY 999U
#define PWM_HALF_DUTY 500U
/* Крок плавного наростання: раз на 10 мс зрушити duty на 8 кроків. */
#define FADE_STEP_MS 10U
#define FADE_STEP 8U

/* Режим чотирьох світлодіодів. */
#define PWM_MODE_OFF 0U
#define PWM_MODE_HALF 1U
#define PWM_MODE_MAX 2U
#define PWM_MODE_FADE 3U

/* Який генератор реально живить SYSCLOCK. */
#define CLOCK_SRC_HSE 0U
#define CLOCK_SRC_HSI 1U

/* Периферія, яку зібрав Cube у main.c. */
extern TIM_HandleTypeDef htim2;
extern TIM_HandleTypeDef htim3;
extern UART_HandleTypeDef huart1;

/* Мілісекунди від старту прошивки. Додає переривання TIM2. */
static volatile uint32_t g_uptime_ms;
/* Поточний режим і скважність, спільні для чотирьох каналів. */
static volatile uint8_t g_pwm_mode = PWM_MODE_OFF;
static volatile uint16_t g_pwm_duty;
/* Напрямок плавного ходу: +1 угору, -1 униз. */
static volatile int8_t g_fade_dir = 1;
/* Лічильник мілісекунд усередині одного кроку fade. */
static volatile uint8_t g_fade_tick;

/* Байт, у який HAL кладе черговий прийнятий символ. */
static uint8_t g_rx_byte;
/* Зібраний рядок до символу нового рядка. */
static char g_rx_line[UART_LINE_MAX];
static uint8_t g_rx_len;
/* 1 – рядок готовий, його забере головний цикл. */
static volatile uint8_t g_line_ready;
/* 0 – кварц HSE, 1 – запасний внутрішній HSI. */
static uint8_t g_clock_src = CLOCK_SRC_HSE;

/*
 * HSE не запустився: немає кварцу, непропай або частота не 25 МГц.
 * HAL_RCC_OscConfig чекає свій тайм-аут і повертає помилку.
 * Після скидання SYSCLOCK ще на HSI, тому PLL можна вимкнути
 * і зібрати ті самі 96 МГц від внутрішнього генератора: 16 МГц / 8 * 96 / 2.
 */
void App_ClockFallbackHsi(void)
{
    RCC_OscInitTypeDef osc = {0};

    /* Спочатку лише вмикаємо HSI і гасимо PLL, щоб перемкнути його джерело. */
    osc.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    osc.HSISState = RCC_HSI_ON;
    osc.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
```

```

    osc.PLL.PLLState = RCC_PLL_OFF;
    if (HAL_RCC_OscConfig(&osc) != HAL_OK) {
        Error_Handler();
    }

    /* 16 МГц / 8 = 2 МГц на вході PLL, * 96 / 2 = 96 МГц SYSCLK. */
    osc = (RCC_OscInitTypeDef){0};
    osc.OscillatorType = RCC_OSCILLATORTYPE_HSI;
    osc.HSISState = RCC_HSI_ON;
    osc.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    osc.PLL.PLLState = RCC_PLL_ON;
    osc.PLL.PLLSource = RCC_PLLSOURCE_HSI;
    osc.PLL.PLLM = 8;
    osc.PLL.PLLN = 96;
    osc.PLL.PLLP = RCC_PLLP_DIV2;
    osc.PLL.PLLQ = 4;
    if (HAL_RCC_OscConfig(&osc) != HAL_OK) {
        Error_Handler();
    }

    g_clock_src = CLOCK_SRC_HSI;
}

/* Кварц і PLL зібрались штатно. */
void App_ClockNoteHse(void)
{
    g_clock_src = CLOCK_SRC_HSE;
}

/* Один рядок у USART1, без переводу рядка. Тайм-аут передачі 100 мс. */
static void uart_send(const char *s)
{
    HAL_UART_Transmit(&huart1, (uint8_t *)s, (uint16_t)strlen(s), 100);
}

/* Рядок відповіді: текст і CR LF, як чекає термінал ведучого. */
static void uart_send_line(const char *s)
{
    uart_send(s);
    uart_send("\r\n");
}

/* Одна скважність на PA6, PA7, PB0, PB1. Вище максимуму обрізаємо. */
static void pwm_set_all(uint16_t duty)
{
    if (duty > PWM_MAX_DUTY) {
        duty = PWM_MAX_DUTY;
    }
    g_pwm_duty = duty;
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, duty);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, duty);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, duty);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_4, duty);
}

/* Вмикає один із чотирьох режимів і одразу виставляє початкову скважність. */
static void pwm_apply_mode(uint8_t mode)
{
    g_pwm_mode = mode;
    if (mode == PWM_MODE_OFF) {
        pwm_set_all(0);
    } else if (mode == PWM_MODE_HALF) {
        pwm_set_all(PWM_HALF_DUTY);
    } else if (mode == PWM_MODE_MAX) {
        pwm_set_all(PWM_MAX_DUTY);
    } else if (mode == PWM_MODE_FADE) {
        /* Fade починається з нуля і йде вгору. */
        g_fade_dir = 1;
    }
}

```

```

    g_fade_tick = 0;
    pwm_set_all(0);
}
}

/* Літери команди до верхнього регістру, щоб «ping» і «PING» збігались. */
static void str_up(char *s)
{
    for (; *s; s++) {
        *s = (char)toupper((unsigned char)*s);
    }
}

/* Після скидання ведений сам повідомляє, що живий, і чим тактується. */
static void send_banner(void)
{
    char clk[UART_LINE_MAX];

    uart_send_line("INFO STM32F411 slave started");
    if (g_clock_src == CLOCK_SRC_HSE) {
        snprintf(clk, sizeof(clk),
            "INFO clock: HSE 25 MHz, SYSCLOCK %lu Hz, PCLK2 %lu Hz",
            (unsigned long)HAL_RCC_GetSysClockFreq(),
            (unsigned long)HAL_RCC_GetPCLK2Freq());
    } else {
        snprintf(clk, sizeof(clk),
            "INFO clock: HSI fallback, SYSCLOCK %lu Hz, PCLK2 %lu Hz",
            (unsigned long)HAL_RCC_GetSysClockFreq(),
            (unsigned long)HAL_RCC_GetPCLK2Freq());
    }
    /* HSI гірше тримає частоту, ніж кварц, тому baud може поплисти. */
    uart_send_line("INFO clock WARNING: HSE failed, UART baud less accurate");
}
uart_send_line(clk);
uart_send_line("INFO PWM LEDs: PA6 PA7 PB0 PB1");
uart_send_line("INFO commands:");
uart_send_line("INFO   PWM MAX | PWM OFF | PWM HALF | PWM FADE");
uart_send_line("INFO   TIME");
uart_send_line("INFO   STATUS");
uart_send_line("INFO   PING");
uart_send_line("INFO   HELP");
uart_send_line("OK READY");
}

/* Ім'я режиму для рядка STATUS. */
static const char *pwm_mode_name(void)
{
    if (g_pwm_mode == PWM_MODE_HALF) {
        return "HALF";
    }
    if (g_pwm_mode == PWM_MODE_MAX) {
        return "MAX";
    }
    if (g_pwm_mode == PWM_MODE_FADE) {
        return "FADE";
    }
    return "OFF";
}

/* Розбір одного рядка. Відповідь завжди один рядок OK або ERR. */
static void handle_line(char *line)
{
    char cmd[16] = {0};
    char al[16] = {0};
    int n;

    str_up(line);

    /* Перше слово – команда, друге – аргумент (для PWM). */

```

```

    n = sscanf(line, "%15s %15s", cmd, a1);
    if (n < 1) {
        uart_send_line("ERR empty");
        return;
    }

    if (strcmp(cmd, "PING") == 0) {
        uart_send_line("OK PONG");
        return;
    }
    if (strcmp(cmd, "HELP") == 0) {
        uart_send_line("OK PWM MAX|OFF|HALF|FADE | TIME | STATUS | PING | HELP");
        return;
    }
    if (strcmp(cmd, "TIME") == 0) {
        char out[UART_LINE_MAX];
        uint32_t ms = g_uptime_ms;
        uint32_t s = ms / 1000U;
        uint32_t h = s / 3600U;
        uint32_t m = (s / 60U) % 60U;
        uint32_t sec = s % 60U;
        /* Локальний час від Reset, без шкали ведучого. */
        snprintf(out, sizeof(out),
            "OK TIME LOCAL_MS=%lu UPTIME=%02lu:%02lu:%02lu",
            (unsigned long)ms, (unsigned long)h, (unsigned long)m, (unsigned long)sec);
        uart_send_line(out);
        return;
    }
    if (strcmp(cmd, "STATUS") == 0) {
        char out[UART_LINE_MAX];
        snprintf(out, sizeof(out), "OK STATUS PWM=%s DUTY=%u LOCAL_MS=%lu CLK=%s",
            pwm_mode_name(), (unsigned)g_pwm_duty, (unsigned long)g_uptime_ms,
            (g_clock_src == CLOCK_SRC_HSE) ? "HSE" : "HSI");
        uart_send_line(out);
        return;
    }
    if (strcmp(cmd, "PWM") == 0) {
        if (n < 2) {
            uart_send_line("ERR PWM MAX|OFF|HALF|FADE");
            return;
        }
        if (strcmp(a1, "MAX") == 0) {
            pwm_apply_mode(PWM_MODE_MAX);
            uart_send_line("OK PWM MAX");
            return;
        }
        if (strcmp(a1, "OFF") == 0) {
            pwm_apply_mode(PWM_MODE_OFF);
            uart_send_line("OK PWM OFF");
            return;
        }
        if (strcmp(a1, "HALF") == 0) {
            pwm_apply_mode(PWM_MODE_HALF);
            uart_send_line("OK PWM HALF");
            return;
        }
        if (strcmp(a1, "FADE") == 0) {
            pwm_apply_mode(PWM_MODE_FADE);
            uart_send_line("OK PWM FADE");
            return;
        }
        uart_send_line("ERR PWM MAX|OFF|HALF|FADE");
        return;
    }

    uart_send_line("ERR unknown");
}

```

```

void App_Start(void)
{
    /* Cube лишив період TIM3 максимальним. Для ШІМ потрібні 1000 кроків. */
    __HAL_TIM_SET_AUTORELOAD(&htim3, PWM_ARR);
    /* Чотири канали: PA6, PA7, PB0, PB1. */
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_2);
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3);
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_4);
    pwm_apply_mode(PWM_MODE_OFF);

    /* TIM2: переривання раз на мілісекунду, звітти uptime і крок fade. */
    HAL_TIM_Base_Start_IT(&htim2);
    /* Перший байт USART1. Далі прийом перезавантажує колбек. */
    HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1);

    /* Коротка пауза, щоб ведучий встиг відкрити монітор після скидання. */
    HAL_Delay(50);
    send_banner();
}

void App_Poll(void)
{
    char line[UART_LINE_MAX];

    if (!g_line_ready) {
        return;
    }

    /* Копія рядка під забороною переривань, щоб прийом не порвав буфер. */
    __disable_irq();
    memcpy(line, g_rx_line, UART_LINE_MAX);
    g_line_ready = 0;
    __enable_irq();
    handle_line(line);
}

/* Кожен прийнятий байт USART1. Збирає рядок до \n. \r відкидає. */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    uint8_t ch;

    if (huart->Instance != USART1) {
        return;
    }

    ch = g_rx_byte;
    if (ch == '\r') {
        /* Порожній крок: кінець рядка Windows не входить у команду. */
    } else if (ch == '\n') {
        g_rx_line[g_rx_len] = 0;
        g_line_ready = 1;
        g_rx_len = 0;
    } else if (g_rx_len + 1U < UART_LINE_MAX) {
        g_rx_line[g_rx_len++] = (char)ch;
    } else {
        /* Занадто довгий рядок скидаємо, щоб не вилізли з буфера. */
        g_rx_len = 0;
    }
    HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1);
}

/* Помилка кадру або шуму на лінії: прийом інакше зупиняється. */
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART1) {
        HAL_UART_Receive_IT(&huart1, &g_rx_byte, 1);
    }
}

```

```

}

/* Раз на 1 мс. Рахує час від старту і, у режимі FADE, рухає скважність. */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    int32_t duty;

    if (htim->Instance != TIM2) {
        return;
    }

    g_uptime_ms++;

    if (g_pwm_mode != PWM_MODE_FADE) {
        return;
    }

    if (++g_fade_tick < FADE_STEP_MS) {
        return;
    }
    g_fade_tick = 0;

    duty = (int32_t)g_pwm_duty + (g_fade_dir * (int32_t)FADE_STEP);
    if (duty >= (int32_t)PWM_MAX_DUTY) {
        duty = (int32_t)PWM_MAX_DUTY;
        g_fade_dir = -1;
    } else if (duty <= 0) {
        duty = 0;
        g_fade_dir = 1;
    }
    pwm_set_all((uint16_t)duty);
}

```

Короткі пояснення

- 1) Консоль ESP32-S3 — USB Serial/JTAG, тим самим кабелем, що й прошивка. Канал до STM32 — UART2 GPIO17 TX / GPIO16 RX. На STM32 консолі немає; USART1 лише для команд.
- 2) Рядок + \n — кадр протоколу. ESP32 шле, чекає рядок OK або ERR. Це логічний half-duplex на окремих TX і RX.
- 3) PWM MAX|OFF|HALF|FADE. STATUS і TIME показують LOCAL_MS веденого. SYNC і STATS рахує лише ESP32.
- 4) Клавіатура монітора читається з stdin цієї консолі. UART0 на S3 до кабеля не підключений: запис у нього дає таймаут Monitor.